Programación Orientada a Objetos

Paradigmas y Lenguajes de Programación

Punteros y Gestión Dinámica de Memoria

Introducción

- **¿Qué es la Programación?**
  - Proceso de diseñar y construir un programa ejecutable para lograr un objetivo computacional específico.
- **¿Por qué son Importantes los Paradigmas y Lenguajes?**
  - Determinan cómo pensamos y resolvemos problemas con el código.
  - Influyen en la eficiencia, mantenibilidad y escalabilidad del software.
- **Objetivos de la Presentación:**
  - Comprender los diferentes paradigmas de programación.
  - Analizar las características de los lenguajes.
  - Distinguir entre compiladores e intérpretes.

Punteros y Gestión Dinámica de Memoria

Paradigmas de Programación - Definición

- **¿Qué es un Paradigma de Programación?**
  - Un **estilo fundamental** de programación, una forma de conceptualizar y construir soluciones de software.
  - No es un lenguaje, sino un **modelo abstracto** o un conjunto de principios para organizar el código y el pensamiento.
  - Diferentes "filosofías" para resolver problemas.
- **Analogía:** Como un chef que elige entre cocinar a la parrilla, al horno o salteado; cada método tiene sus propias herramientas y enfoques.

Punteros y Gestión Dinámica de Memoria

Tipos de Paradigmas

- **Clasificación Principal:**
  - **Imperativo:** Se centra en "cómo" hacer las cosas, dando instrucciones paso a paso.
  - **Declarativo:** Se centra en "qué" hacer, describiendo el resultado deseado.
  - **Orientado a Objetos:** Un sub-paradigma que organiza el código alrededor de "objetos" que modelan el mundo real.
- **Mencionar Otros (sin profundizar mucho):**
  - Concurrente
  - Lógico
  - Funcional (será cubierto dentro de declarativo)

Punteros y Gestión Dinámica de Memoria

Paradigma Imperativo o por Procedimientos

- **Definición:**
  - Describe la secuencia de instrucciones que la computadora debe ejecutar para cambiar el estado del programa.
  - El programador define **cómo** el programa debe realizar una tarea.
  - Foco en la **modificación del estado** (variables, memoria).
- **Características:**
  - Uso de variables y asignación.
  - Estructuras de control (bucles for, while, condicionales if/else).
  - Procedimientos o subrutinas (funciones).

```
// Sumar los primeros N números
FUNCION sumarNumeros(N):
    total = 0
    PARA i DESDE 1 HASTA N:
        total = total + i
    RETORNAR total

IMPRIMIR sumarNumeros(5) // Resultado:
15
```

- **Lenguajes Comunes:** C, Fortran, Pascal, COBOL.

Punteros y Gestión Dinámica de Memoria

Paradigma Declarativo - Introducción

- **Definición:**
  - Describe **qué** problema se debe resolver, sin especificar explícitamente **cómo** resolverlo.
  - El programador se enfoca en la lógica del programa, no en el flujo de control.
  - La ejecución es controlada por el sistema subyacente.
- **Tipos Principales:**
  - Funcional
  - Lógico
  - De Reglas de Producción

Punteros y Gestión Dinámica de Memoria

Paradigma Declarativo - Funcional

- **Definición**
  - Trata la computación como la evaluación de funciones matemáticas.
  - Se enfoca en la inmutabilidad de los datos y la ausencia de efectos secundarios.
  - Las funciones son "ciudadanos de primera clase" (pueden pasarse como argumentos, retornarse, etc.).
- **Características:**
  - Funciones puras (mismo input -> mismo output, sin efectos secundarios).
  - Recursión en lugar de bucles.
  - Inmutabilidad.

```
# Sumar los primeros N números usando
recursión
def sumar_numeros_recursivo(n):
    if n == 0:
        return 0
    else:
        return n + sumar_numeros_recursivo(n
- 1)

print(sumar_numeros_recursivo(5)) #
Resultado: 15
```

- **Lenguajes Comunes:** Haskell, Lisp, Erlang, Scala (híbrido), Python (soporte funcional).

Punteros y Gestión Dinámica de Memoria

Paradigma Declarativo - Lógico

- **Definición:**
  - Basado en la lógica formal. El programa es un conjunto de hechos y reglas.
  - El motor de inferencia del lenguaje busca automáticamente soluciones a las consultas.
- **Características:**
  - Hechos (afirmaciones básicas).
  - Reglas (inferencias lógicas).
  - Consultas (preguntas al sistema).

```
% Hechos
padre(juan, pedro).
padre(pedro, ana).
madre(maria, pedro).

% Regla: X es hijo de Y si Y es padre de X o Y es madre
de X
hijo(X, Y) :- padre(Y, X).
hijo(X, Y) :- madre(Y, X).

% Consulta: ¿Quién es hijo de Pedro?
% ?- hijo(X, pedro).
% X = ana.
```

- **Lenguajes Comunes:** Prolog, Datalog.

Punteros y Gestión Dinámica de Memoria

Paradigma Declarativo - De Reglas de Producción

- **Definición:**
  - Sistemas basados en reglas que aplican transformaciones o acciones cuando ciertas condiciones son verdaderas.
  - Utilizados en sistemas expertos y motores de reglas.
- **Características:**
  - Conjunto de reglas "SI (condición) ENTONCES (acción)".
  - Memoria de trabajo (facts).
  - Motor de inferencia que selecciona y dispara reglas.

```
// Sistema de recomendación simple
REGLA 1:
    SI temperatura > 25 Y pronostico == "soleado"
    ENTONCES recomendar("ir a la playa")

REGLA 2:
    SI temperatura < 10 O pronostico == "lluvia"
    ENTONCES recomendar("quedarse en casa")

// Hechos:
temperatura = 28
pronostico = "soleado"

// Resultado: "ir a la playa"
```

- **Lenguajes/Sistemas Comunes:** CLIPS, Drools.

Punteros y Gestión Dinámica de Memoria

Paradigma Orientado a Objetos (POO)

- **Definición:**
  - Un paradigma que organiza el software en torno a **objetos** en lugar de funciones y lógica.
  - Los objetos combinan **datos** (atributos) y **comportamiento** (métodos).
  - Busca modelar el mundo real de una manera más intuitiva.

- **Conceptos Clave:**
  - **Clase:** Plantilla o blueprint para crear objetos (ej. Coche).
  - **Objeto:** Instancia de una clase (ej. miCoche = new Coche()).
  - **Herencia:** Una clase puede heredar propiedades y métodos de otra (reuso de código).
  - **Polimorfismo:** Objetos de diferentes clases pueden responder a la misma llamada de método de manera diferente.
  - **Encapsulamiento:** Ocultamiento de los detalles internos de un objeto, exponiendo solo lo necesario.
  - **Abstracción:** Representar características esenciales sin incluir los detalles de implementación.
- **Lenguajes Comunes:** Java, C++, C#, Python, Ruby, Smalltalk, JavaScript (soporte POO).

Punteros y Gestión Dinámica de Memoria

Punteros como Argumentos de Funciones: Paso por referencia

Característica	Imperativo	Declarativo	Orientado a Objetos
Enfoque	Cómo hacer	Qué hacer	Objetos y su interacción
Control de Flujo	Explícito (bucles, ifs)	Implícito (motor, reglas)	Interacción de métodos
Estado	Modificable (variables)	Inmutable (preferido)	Encapsulado en objetos
Reusabilidad	Procedimientos	Funciones/Reglas	Clases, Herencia
Mantenibilidad	Puede ser compleja	Generalmente alta	Alta (modularidad)
Complejidad	Media	Varía	Media a alta (curva de aprendizaje)
Ejemplos	C, Fortran	Haskell, Prolog, SQL	Java, Python, C++

Punteros y Gestión Dinámica de Memoria

Entornos de Programación Visual

- **Definición:**
  - IDE (Integrated Development Environment) que permite el desarrollo de software utilizando **interfaces gráficas** y componentes visuales.
  - En lugar de escribir todo el código a mano, se arrastran y sueltan elementos, y el entorno genera el código subyacente.
- **Características:**
  - Diseñadores de interfaz de usuario (UI).
  - Eventos y manejo de acciones visuales.
  - Generación automática de código.
- **Ventajas:**
  - Desarrollo rápido de aplicaciones (RAD).
  - Facilita la creación de interfaces de usuario complejas.
  - Curva de aprendizaje inicial más suave para UI.
- **Desventajas:**
  - Menos control granular sobre el código.
  - El código generado puede ser más "verboso" o menos optimizado.
  - Dependencia del entorno.
- **Ejemplos:** Visual Studio (para C#, VB.NET), Android Studio (para Kotlin/Java UIs), Xcode (para Swift/Objective-C UIs), NetBeans (para Java Swing/FX).

Punteros y Gestión Dinámica de Memoria

Lenguaje de Programación - Definición

- **¿Qué es un Lenguaje de Programación?**
  - Un **idioma artificial** diseñado para expresar computaciones que pueden ser realizadas por una máquina (computadora).
  - Consiste en un conjunto de **reglas sintácticas** (cómo se escribe) y **semánticas** (qué significa).
  - Sirven como puente entre el pensamiento humano y la ejecución de la máquina.
- **Analogía:** Es el idioma que usamos para dar instrucciones claras y precisas a la computadora, al igual que hablamos un idioma para comunicarnos entre humanos.

Punteros y Gestión Dinámica de Memoria

Lenguaje de Programación - Características

- **Sintaxis:**
  - Las reglas de cómo se deben combinar los símbolos y palabras clave para formar expresiones y declaraciones válidas. (Ej: ; al final de línea en C++, sangría en Python).
- **Semántica:**
  - El significado de las expresiones y declaraciones. Qué acción realiza cada instrucción.
- **Paradigma(s) que soporta:**
  - Muchos lenguajes modernos son **multi-paradigma** (ej: Python, JavaScript, C++).
- **Tipado (Estático/Dinámico, Fuerte/Débil):**
  - **Estático:** El tipo de una variable se verifica en tiempo de compilación (ej: Java, C++).
  - **Dinámico:** El tipo se verifica en tiempo de ejecución (ej: Python, JavaScript).
  - **Fuerte:** No permite conversiones de tipo implícitas que puedan llevar a errores (ej: Python).
  - **Débil:** Permite conversiones implícitas (ej: JavaScript).
- **Orientado a un Dominio Específico (DSL) vs. Propósito General (GPL):**
  - **DSL:** SQL (consultas a bases de datos), HTML (estructurar web).
  - **GPL:** Python, Java, C++ (sirven para muchos tipos de aplicaciones).

Punteros y Gestión Dinámica de Memoria

Elementos de un Lenguaje de Programación

- **Palabras Clave (Keywords):**
  - Palabras reservadas con un significado especial (ej: if, else, for, class, function).
- **Identificadores:**
  - Nombres dados a variables, funciones, clases, etc. (ej: miVariable, calcularSuma).
- **Operadores:**
  - Símbolos que realizan operaciones (ej: +, -, *, /, =, ==, <).
- **Literales:**
  - Valores fijos en el código (ej: 10, "Hola Mundo", 3.14).
- **Comentarios:**
  - Texto ignorado por el procesador, para documentación y explicación (ej: // comentario o # comentario).
- **Estructuras de Control:**
  - Definen el flujo de ejecución del programa (condicionales, bucles).
- **Estructuras de Datos:**
  - Formas de organizar y almacenar datos (arrays, listas, diccionarios, objetos).

Punteros y Gestión Dinámica de Memoria

Procesadores de Lenguaje - Compiladores

- **Definición:**
  - Un programa que traduce el código fuente (escrito por el programador) en código máquina o código intermedio (bytecode) **antes** de la ejecución.
  - La traducción se realiza **una sola vez**.
- **Proceso:**
  - **Código Fuente** (.c, .java)
  - **Compilador**
  - **Código Objeto** (.o, .class)
  - **Enlazador (Linker)**
  - **Ejecutable** (.exe)
- **Ventajas:**
  - **Rendimiento:** El código ejecutable es generalmente más rápido.
  - **Detección de Errores:** Muchos errores se detectan en tiempo de compilación.
  - **Independencia del Código Fuente:** El ejecutable puede distribuirse sin el código fuente.
- **Desventajas:**
  - Requiere un paso de compilación antes de cada ejecución.
  - Menos flexibilidad para cambios rápidos.
- **Ejemplos:** C, C++, Java (compila a bytecode, luego JVM interpreta/compila JIT).

Punteros y Gestión Dinámica de Memoria

Procesadores de Lenguaje - Intérpretes

- **Definición:**
  - Un programa que **ejecuta directamente** las instrucciones del código fuente, línea por línea, sin una traducción previa a un ejecutable completo.
  - La traducción y ejecución se realizan **simultáneamente**.
- **Proceso:**
  - **Código Fuente** (.py, .js)
  - **Intérprete**
  - **Ejecución Línea a Línea**
- **Ventajas:**
  - **Flexibilidad:** Facilita la depuración y los cambios rápidos.
  - **Portabilidad:** El mismo código fuente puede ejecutarse en diferentes plataformas si el intérprete está disponible.
  - Ideal para scripting y desarrollo rápido.
- **Desventajas:**
  - **Rendimiento:** Generalmente más lento que el código compilado.
  - **Errores en Tiempo de Ejecución:** Los errores de sintaxis o lógica pueden no ser detectados hasta que esa línea se ejecute.
- **Ejemplos:** Python, JavaScript, Ruby, PHP.

Punteros y Gestión Dinámica de Memoria

Comparación y Limitaciones de Distintos Lenguajes de Programación

Característica	C/C++ (Compilado)	Python (Interpretado)	JavaScript (Interpretado)	Java (Compilado a Bytecode)
Rendimiento	Muy Alto	Moderado (más lento)	Moderado (JIT en navegadores)	Alto (JVM optimizaciones)
Control de Hardware	Alto (bajo nivel)	Bajo	Bajo	Medio
Curva de Aprendizaje	Empinada	Suave	Media	Media a Empinada
Áreas Comunes	Sistemas operativos, juegos, embebidos	IA, web (backend), ciencia de datos	Web (frontend, backend con Node.js)	Aplicaciones empresariales, Android
Manejo de Memoria	Manual	Automático (Garbage Collector)	Automático	Automático (Garbage Collector)
Paradigma	Multi (Imperativo, POO)	Multi (POO, Funcional, Imperativo)	Multi (POO, Funcional, Imperativo)	POO principal (algo funcional en Java 8+)

Punteros y Gestión Dinámica de Memoria

Conclusiones

- Los **paradigmas** son la base conceptual de cómo abordamos la programación.
- Los **lenguajes** son las herramientas concretas que usamos, y su elección impacta directamente en el proyecto.
- **Compiladores e Intérpretes** son fundamentales para el funcionamiento de los lenguajes, cada uno con sus pros y contras.
- Elegir el **paradigma y lenguaje adecuados** es crucial para el éxito de un proyecto de software.
- La Programación Orientada a Objetos es un paradigma potente para modelar sistemas complejos.